

人工智能训练师 理论知识点讲解10

陈老师

2024-10



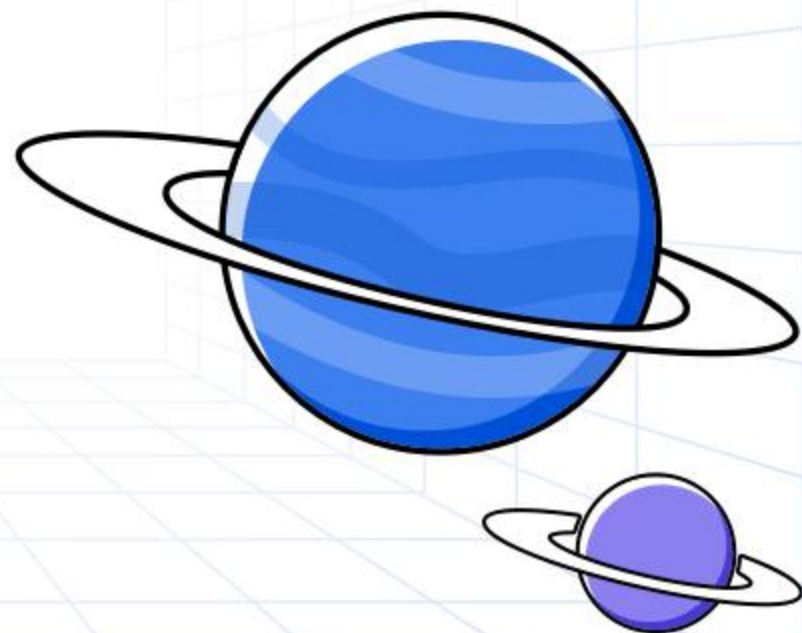
目录

CONTENTS

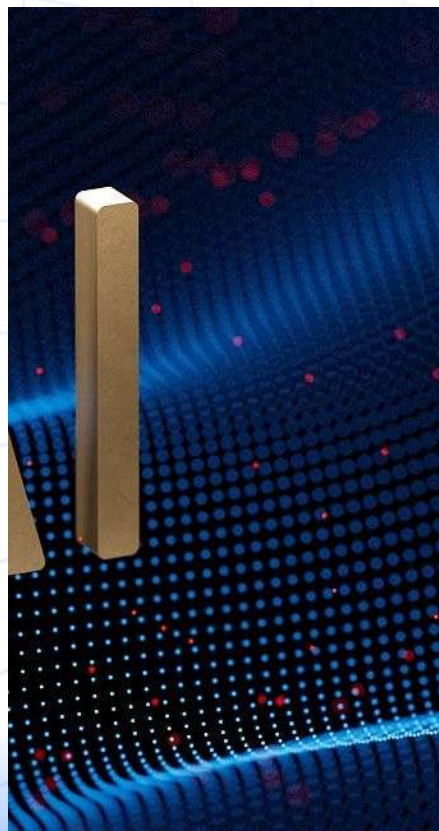
- 软件测试概述
- 等价类划分法详解
- 边界值法详解
- 其他常用测试方法介绍
- 自动化测试工具与框架
- 软件测试实践建议与总结

01

软件测试概述



软件测试定义与目的



定义

软件测试是使用人工或自动的手段来运行或测定某个软件系统的过程，其目的在于检验它是否满足规定的**需求**或弄清预期结果**与实际结果之间的差别**。

目的

确保软件系统的质量和可靠性，以满足用户的需求和期望；发现软件中存在的缺陷、错误和不符合规范的行为；为软件发布提供质量保障。

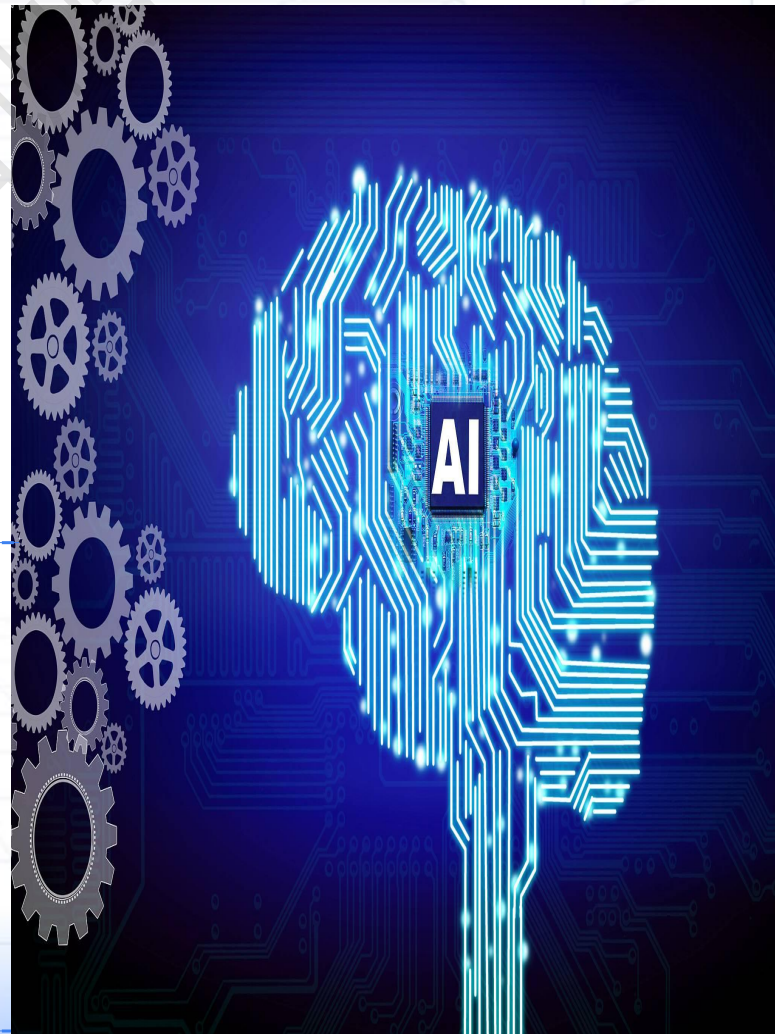
软件测试分类与流程

分类

根据测试方法可分为动态测试和静态测试；根据测试角度可分为白盒测试和黑盒测试；其他分类还包括逻辑功能测试、界面测试、易用性测试、安装测试、兼容性测试等。

流程

通常包括测试计划制定、测试用例设计、测试环境搭建、测试执行、缺陷跟踪与回归测试等环节。



静态测试与动态测试

静态测试

不运行被测程序，通过分析和检查程序代码、结构、过程等来发现错误。

应用示例

静态测试过程包括代码审查、单元测试、集成测试和系统测试等。测试人员可以借助专门的工具辅助测试，如代码分析工具、代码覆盖率工具等。

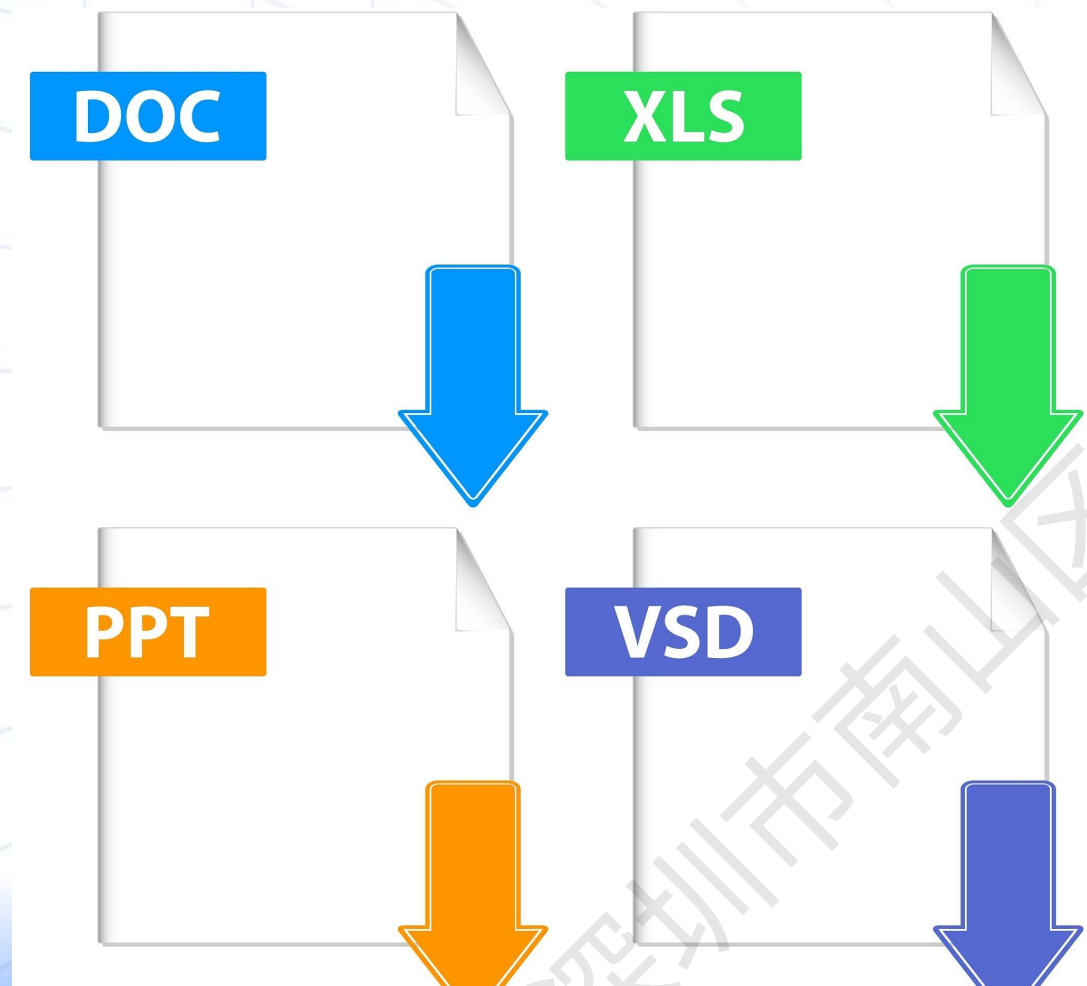
动态测试

通过运行被测程序，检查运行结果与预期结果的差异，从而发现错误。

应用示例

动态测试包括负载测试、压力测试、恢复测试和兼容性测试等。测试人员可以通过模拟大量用户或实际用户来进行测试，借助性能测试工具、负载测试工具等辅助测试。

黑盒测试



黑盒测试

将程序看作一个黑盒子，只关注输入和输出，不考虑程序内部结构和实现。

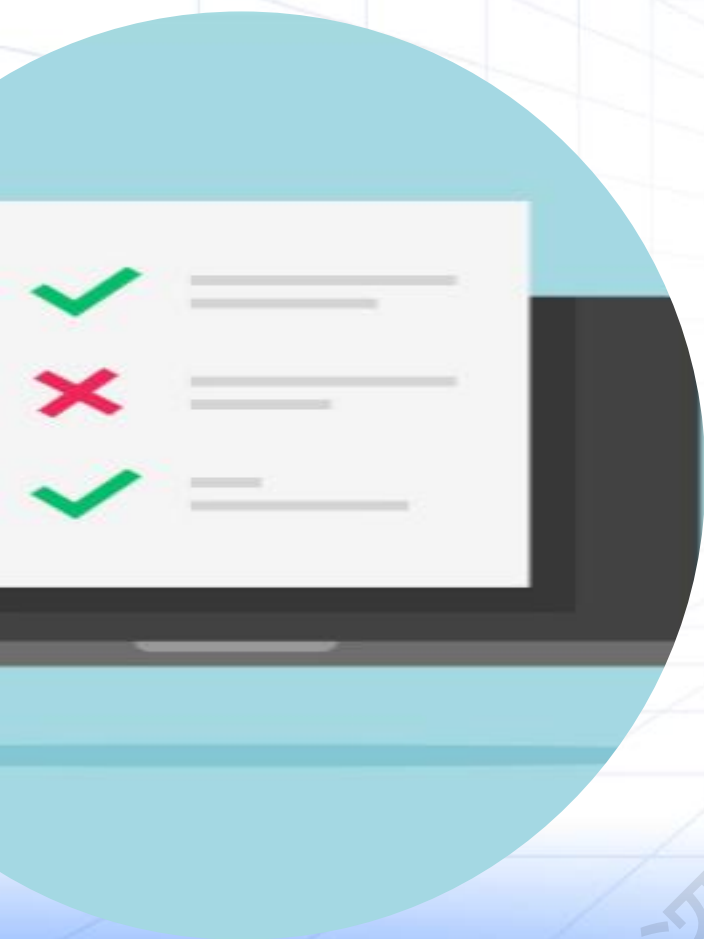
等价类划分

将输入数据划分为若干个等价类，从每个等价类中选取一个或多个代表性数据进行测试。

边界值分析

针对输入和输出的边界值进行测试，容易发现程序中的错误。

白盒测试



白盒测试

了解程序内部结构和实现，针对程序内部逻辑进行测试。

语句覆盖

测试每个可执行语句是否都被执行过。

分支覆盖

测试每个分支是否都被执行过，包括真分支和假分支。

灰盒测试

灰盒测试

介于黑盒测试和白盒测试之间，或者说是两者的结合，既关注输入和输出，也关注程序内部部分实现。灰盒测试可以避免过度测试，精简冗余用例。



接口测试

针对程序接口进行测试，验证接口功能、稳定性和安全性。

集成测试

测试不同模块之间的交互和集成，确保模块之间能够正确通信和协作。

常用测试方法及原理简介



单元测试

针对软件设计的最小单位——程序模块进行正确性检验的测试工作。以白盒为主，一般由开发人员来完成。



集成测试

在单元测试的基础上把软件系统中所有的模块按照设计要求全部组装起来继续继续测试的过程，会出现很多临时版本（迭代测试）。集成测试主要以白盒为主，一般由开发人员来完成。



确认测试

这一阶段的目的是向未来的用户表明系统能够像预定要求那样工作。经集成测试后，已经按照设计把所有的模块组装成一个完整的软件系统，接口错误也已经基本排除了，接着就应该进一步验证软件的有效性，这就是确认测试的任务，即软件的功能和性能如同用户所合理期待的那样。

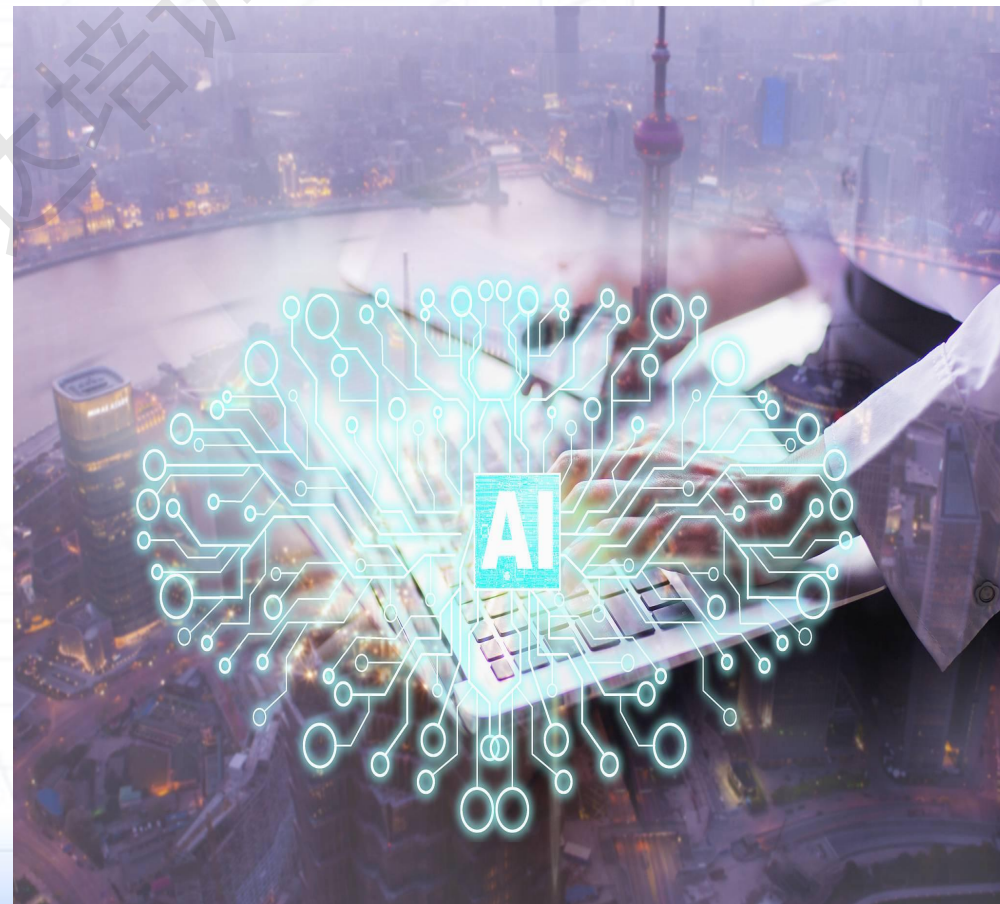
常用测试方法及原理简介

系统测试

在真实的系统工作环境下通过与系统的需求定义作比较,检验完整的软件配置项能否和系统正确连接,发现软件与系统/子系统设计文档和用户需求不符合或与之矛盾的地方。

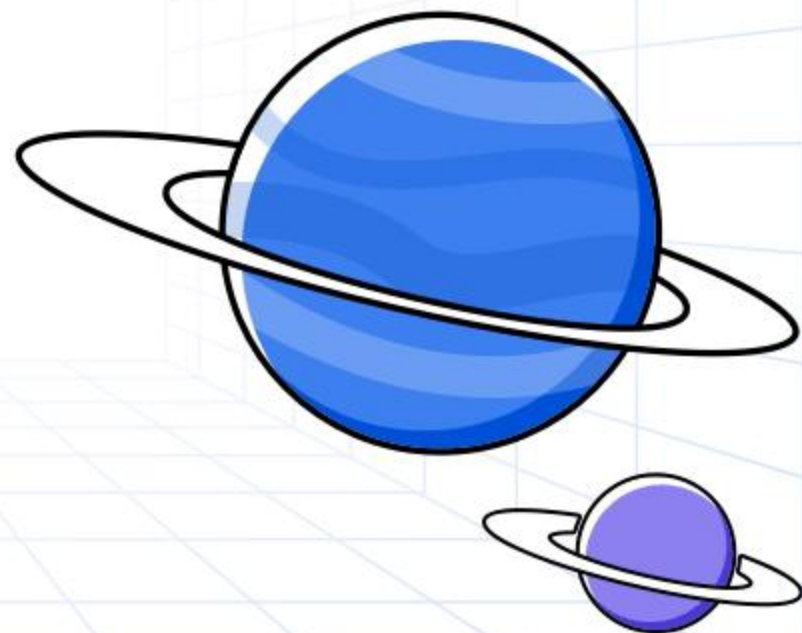
验收测试

验收测试是部署软件之前的最后一个测试操作。验收测试的目的是确保软件准备就绪,并且可以让最终用户将其用于执行软件的既定功能和任务。



02

等价类划分法详解



等价类划分法基本概念

定义

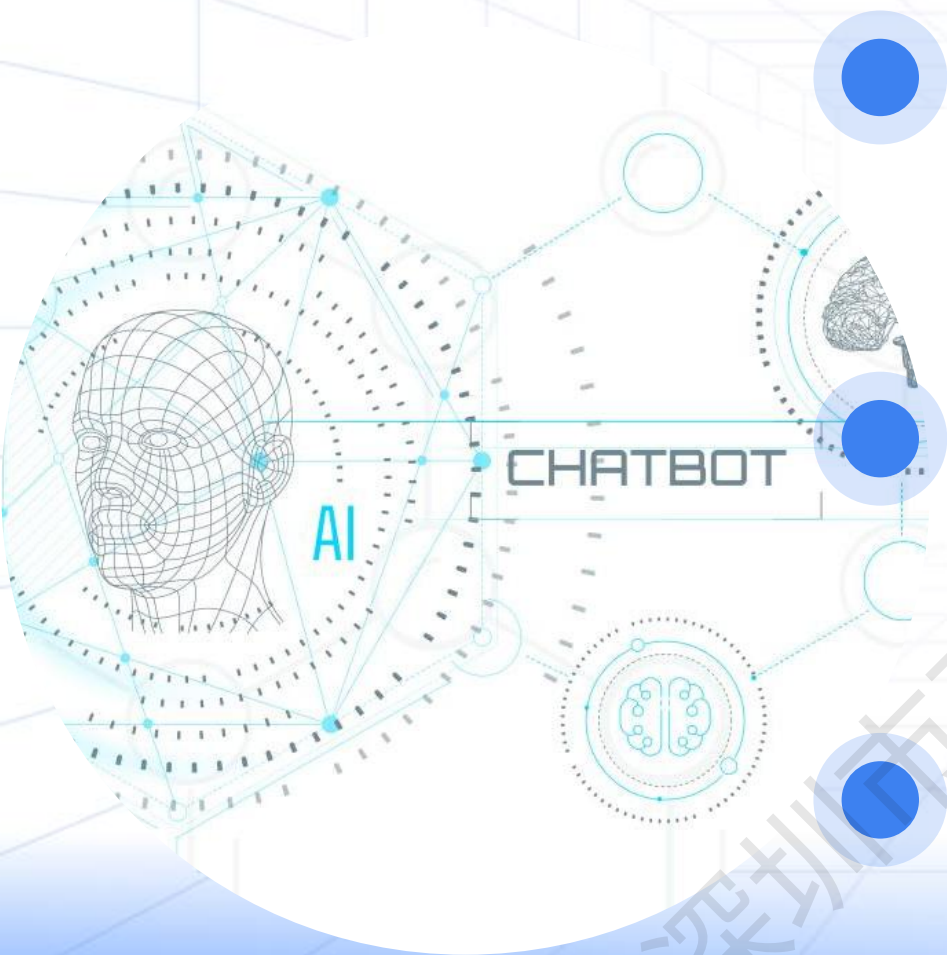
等价类划分是一种黑盒测试方法，它将程序的输入域划分为若干个数据类，然后从每个数据类中选取少数代表性数据进行测试。

目的

通过代表性数据来发现程序中的错误，以此减少测试用例的数量，同时保持测试的有效性。

分类

等价类可分为有效等价类和无效等价类。有效等价类指符合规格说明的、合理的输入数据集合；无效等价类指不符合规格说明的、无意义的输入数据集合。



等价类划分原则与技巧



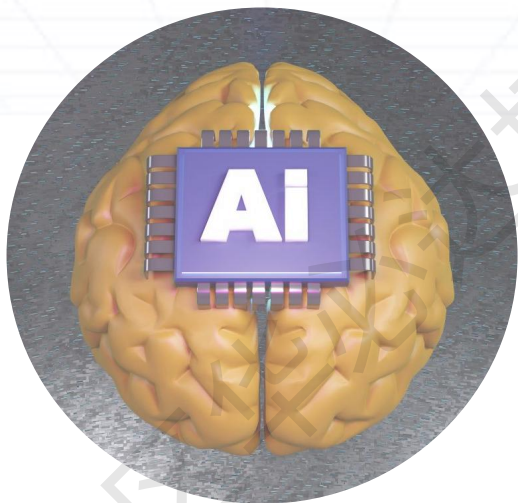
原则

在输入条件规定了取值范围或值的个数时，可以确定一个有效等价类和两个无效等价类。



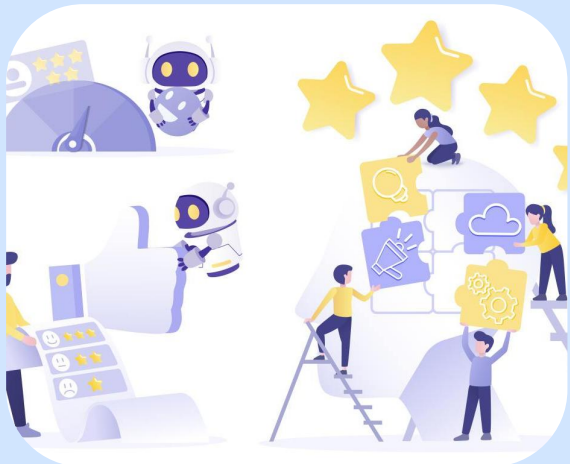
在输入条件规定了输入值的集合或是规定了“必须如何”的条件时，可以确定一个有效等价类和一个无效等价类。

等价类划分原则与技巧



在输入数据是一组值，并且程序要对每一个输入值分别处理的情况下，可为每一个输入值确定一个有效等价类和一个无效等价类。

等价类划分原则与技巧



技巧



仔细分析需求规格说明，找出所有可能的输入条件和限制。



根据输入限制条件，建立等价类表，列出所有有效和无效等价类。

等价类划分原则与技巧



为每一个等价类规定一个唯一的编号。



设计一个新的测试用例，使其尽可能多地覆盖尚未被覆盖地有效等价类，重复这一步，直到**所有的有效等价类都被覆盖**为止。但对于**无效等价类**，**不要求全部覆盖**。

对于无效等价类，由于其数量可能较多，通常选取具有代表性的、可能导致软件出现错误或异常的无效等价类进行测试。

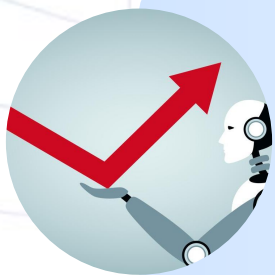


案例分析：某网站登录功能测试

功能描述

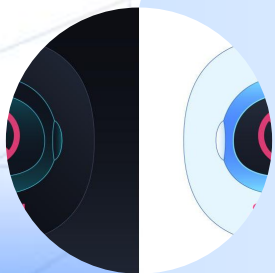
用户通过输入用户名和密码进行登录。

案例分析：某网站登录功能测试



等价类划分

有效等价类



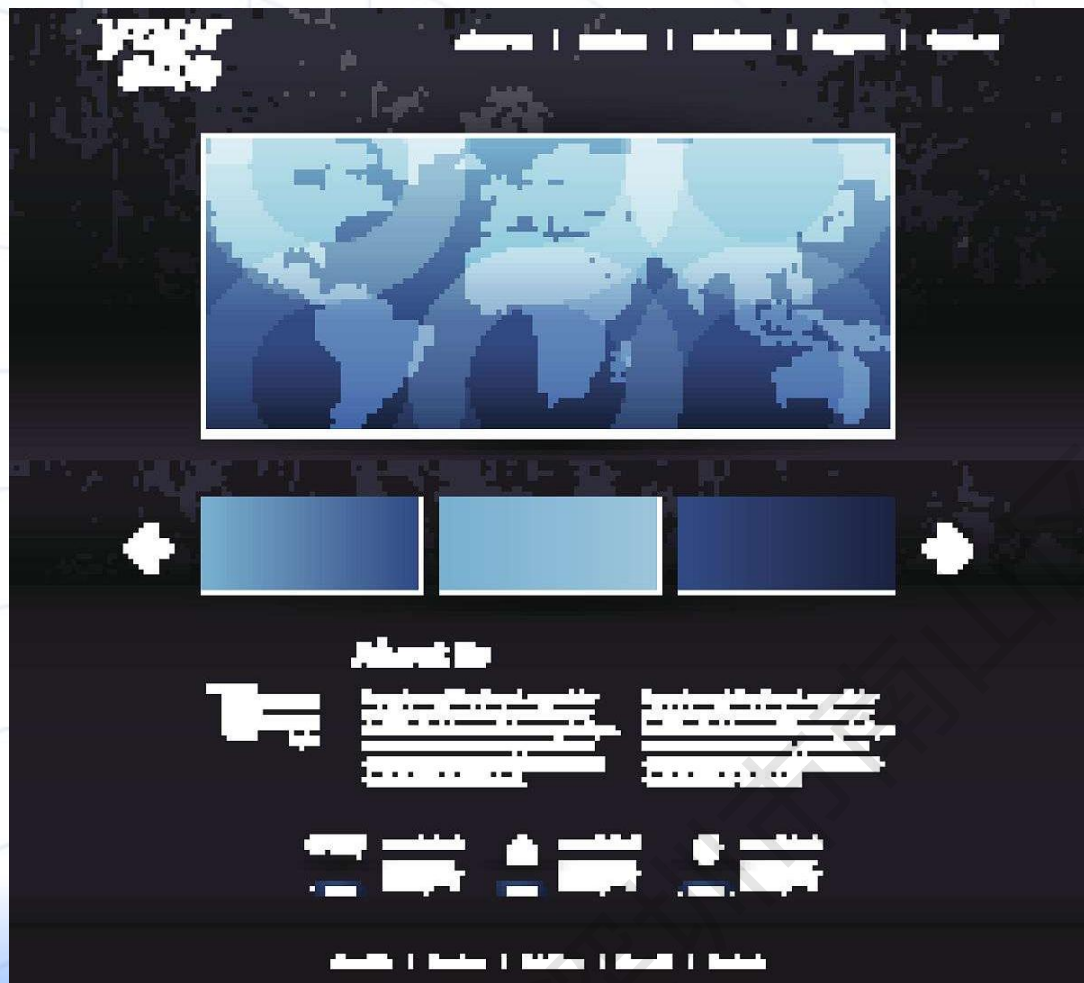
11 用户名和密码均符合要求（如长度、字符类型等）。

案例分析：某网站登录功能测试

- 无效等价类
- 12 用户名或密码为空。
- 13 用户名或密码长度或字符类型不符合要求，或不正确。



案例分析：某网站登录功能测试



测试用例1

输入正确的用户名和密码，预期登录成功。覆盖11。

测试用例2

输入空的用户名或密码，预期登录失败并提示相应错误信息。覆盖12。

测试用例3

输入长度不符合要求的用户名或密码，预期登录失败并提示相应错误信息。覆盖13。

案例分析：某网站登录功能测试



04 测试用例4

输入包含非法字符的用户名或密码，预期登录失败并提示相应错误信息。覆盖13。

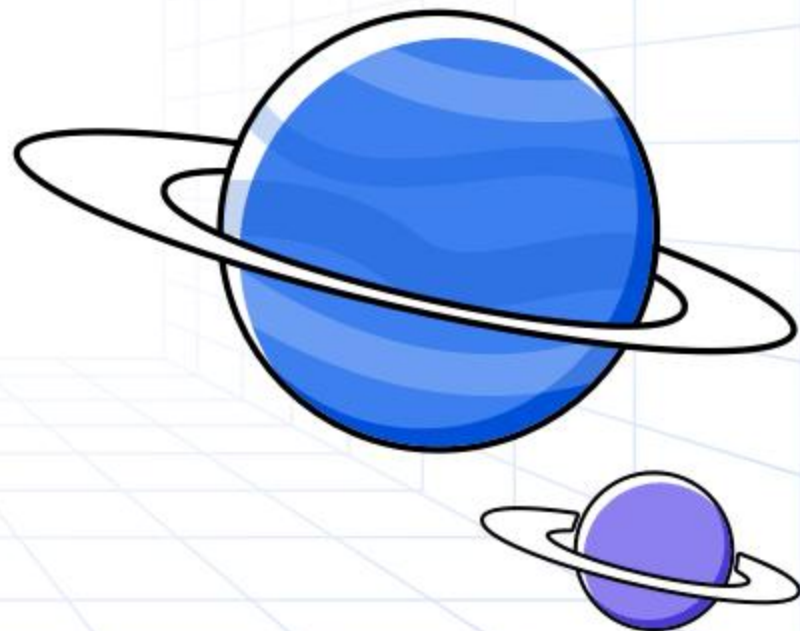


05 测试用例5

输入与数据库中不匹配的用户名或密码，预期登录失败并提示相应错误信息。覆盖13。

03

边界值法详解



边界值法基本概念及原理

基本概念

边界值法是一种在软件测试中常用的方法，主要用于确定输入值的边界和边界之间的测试用例。

原理

其原理基于经验总结，即大量的错误往往发生在输入或输出的边界上。因此，通过针对边界情况设计测试用例，可以有效地发现系统中的错误和漏洞。

边界值选取策略与技巧

01

选取最小值和最大值作为边界值进行测试，这是最基本的边界值选取策略。

02

除了最小值和最大值，还可以选择边界的中间值进行测试，这有助于发现系统在边界附近的错误。

03

测试系统对于非法边界值的处理也是重要的，例如输入超出系统要求范围的值，看系统是否能正确处理。

04

在选择边界值时，还应考虑特殊情况，如输入为空、长度为零、包含特殊字符等，这些情况可能会触发系统的边界处理逻辑。

案例分析：购物车结算功能测试

“

测试场景

以购物车结算功能为例，可以通过边界值法设计测试用例。

深圳市南山区华必达培训学校

案例分析：购物车结算功能测试

购物车商品数量为0时进行结算（边界值
最小值）

购物车商品数量达到系统允许的最大值时进行结算（边界值
最大值）

案例分析：购物车结算功能测试

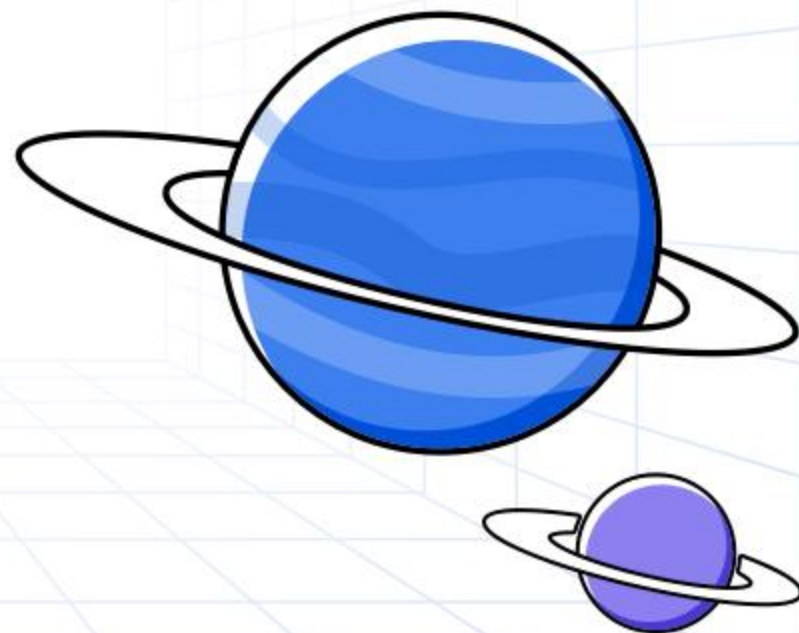
购物车商品数量在系统允许的数量范围中间值时进行结算（边界值中间值）

预期结果与实际结果对比

根据上述测试用例执行测试，并记录预期结果与实际结果进行对比分析。如果发现实际结果与预期结果不符，则说明系统在该边界值处存在问题需要修复。

04

其他常用测试方法 介绍



其他方法

01 因果图法

通过分析输入条件的组合及其对应的输出结果，生成判定表来设计测试用例。

02 错误推测法

基于经验和直觉推测程序中可能存在的错误，有针对性地设计测试用例。

03 场景法

通过模拟用户的实际操作场景来设计测试用例。

04 判定表驱动法

把条件的所有组合罗列出来，针对每种组合确定预期的结果，从而设计测试用例。

05 正交试验法

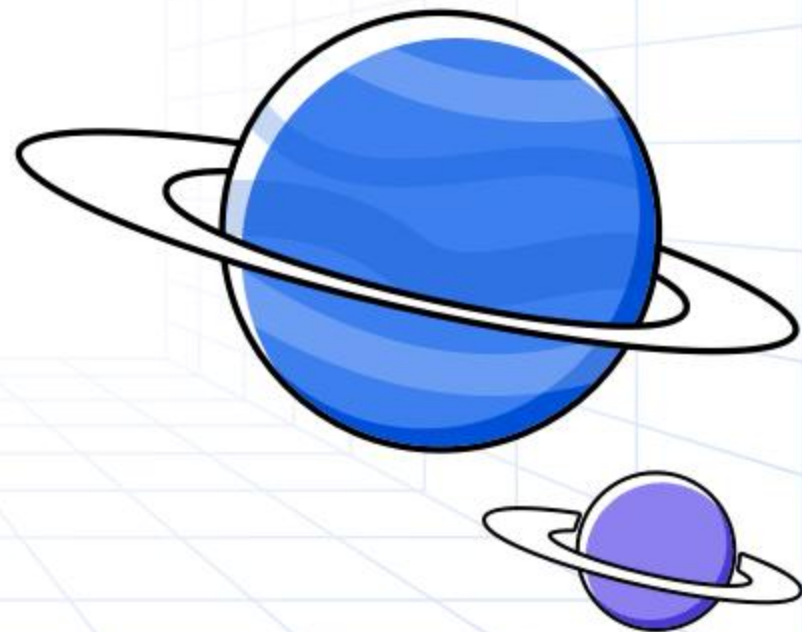
从大量的试验点中挑选适量的、有代表性的点，利用“正交表”合理安排试验的一种科学的试验设计方法。

06 功能图法

用功能图形象地表示程序的功能说明，并生成测试用例。

05

软件测试流程与规范



制定测试计划与方案

确定测试目标

明确测试的目的、范围、重点以及预期结果。

安排测试进度

制定详细的测试时间表，包括各阶段开始和结束时间、关键里程碑等。

制定测试策略

根据软件特点、开发阶段和测试资源，选择合适的测试方法和工具。

分配测试资源

明确测试人员、设备、工具等资源的需求和分配。



设计测试用例与数据准备

分析需求文档

深入理解软件功能和性能要求，确保测试用例覆盖所有关键场景。

设计测试用例

根据测试目标和策略，设计详细的测试用例，包括输入、预期输出和执行条件。

准备测试数据

根据测试用例需求，准备相应的测试数据，包括正常数据、异常数据和边界数据。

编写测试脚本

使用自动化测试工具编写测试脚本，提高测试效率和准确性。



执行测试并记录结果



执行测试用例

按照测试计划和测试用例执行测试，记录实际输出和预期输出的差异。



报告测试结果

将测试结果以图表、报告等形式呈现，便于分析和决策。



跟踪测试进度

实时监控测试进度，确保按计划完成测试任务。



提交缺陷报告

对于发现的缺陷，及时提交缺陷报告，包括缺陷描述、重现步骤和截图等。

缺陷管理及回归测试流程

缺陷跟踪与修复

对提交的缺陷进行跟踪，确保开发团队及时修复并验证修复结果。



回归测试计划

制定回归测试计划，确保修复后的缺陷不再出现，同时验证新功能的正确性。



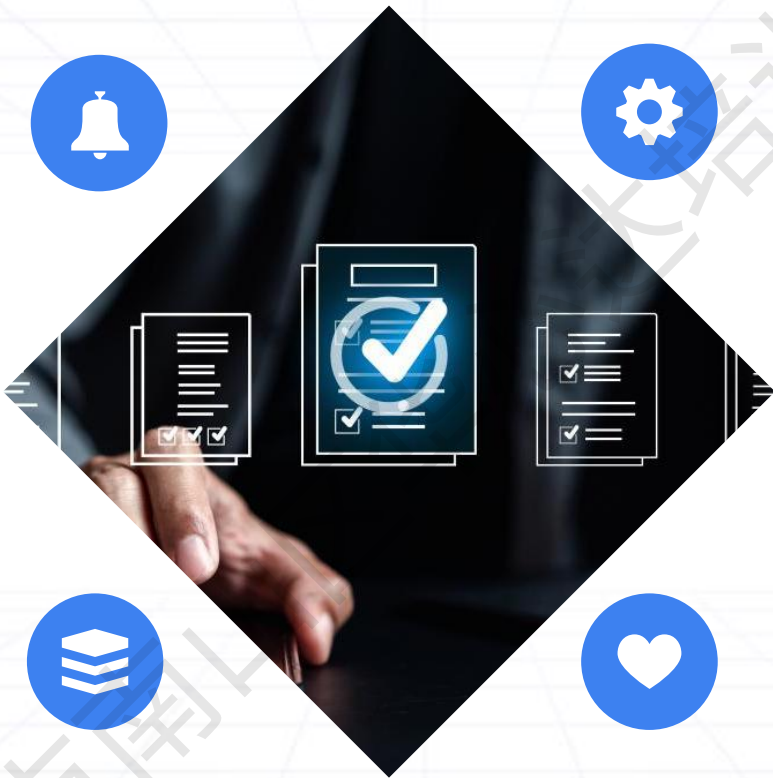
自动化回归测试

利用自动化测试工具执行回归测试，提高测试效率和准确性。



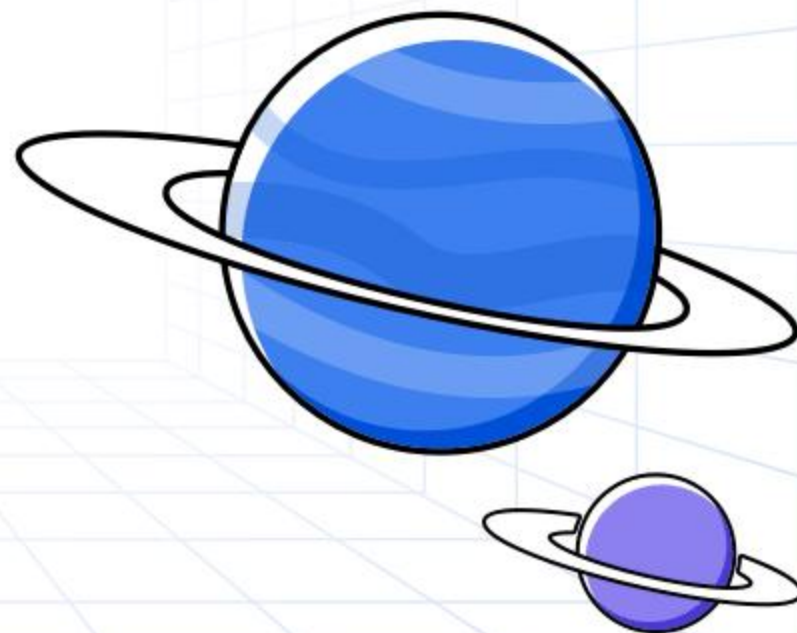
缺陷预防与改进

分析缺陷原因，提出改进措施，预防类似缺陷再次发生。



06

自动化测试工具与 框架



自动化测试概述及优势

自动化测试定义

通过编写程序或脚本来自动执行测试用例，对软件进行验证和测试的过程。

VS

自动化测试优势

提高测试效率，减少人工测试成本；能够执行大量重复性的测试；可以在任何时间进行测试，不受时间和地点的限制；提高测试的准确性和一致性。

常见自动化测试工具介绍

Selenium

一个用于Web应用程序测试的工具。
它支持多种浏览器，包括Chrome、Firefox等，可以录制、编辑和运行自动化测试。



Appium

一个用于移动应用自动化测试的工具。
它支持iOS和Android平台上的原生、混合和移动Web应用程序的测试。

JMeter

一个开源的性能测试工具，可以对Web应用程序进行压力测试和负载测试，以评估系统的性能和稳定性。

自动化测试框架搭建与实现

自动化测试框架的组成

包括测试管理、测试用例设计、测试数据准备、测试执行和测试报告生成等模块。

搭建自动化测试框架的步骤

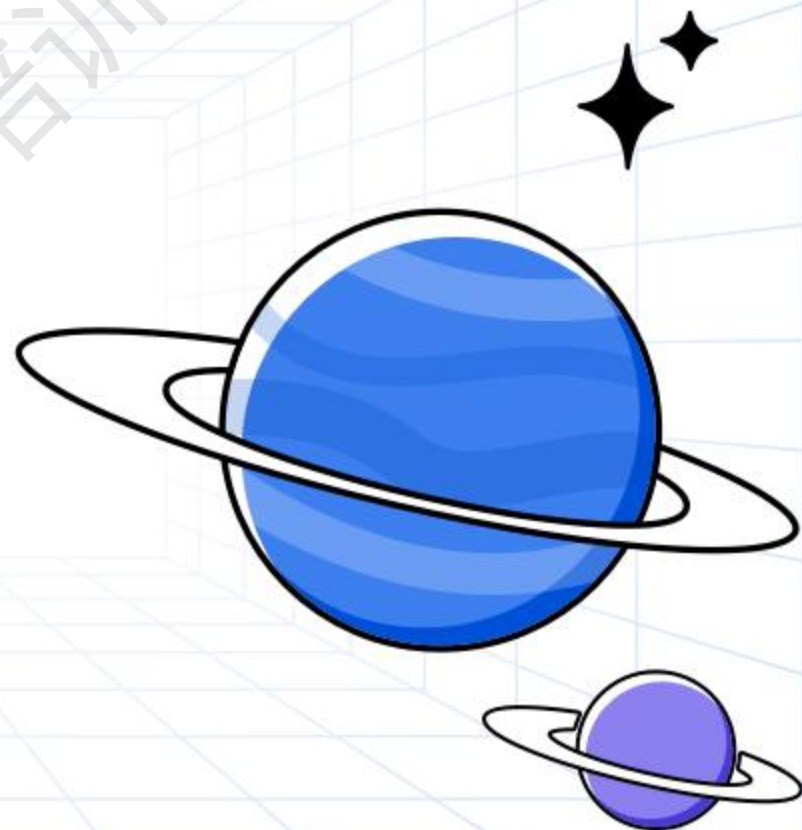
确定测试目标和范围；选择合适的自动化测试工具；设计测试用例和测试数据；编写自动化测试脚本；配置测试环境并执行测试；分析测试结果并生成测试报告。

实现自动化测试框架的注意事项

要确保测试脚本的可读性和可维护性；要对测试数据进行有效管理，以便重复使用；要关注测试执行的稳定性和可靠性，避免出现误报或漏报的情况。

07

软件测试实践建议 与总结



提高软件测试效率和质量的方法

自动化测试

通过自动化测试工具，可以快速执行测试用例，提高测试效率，并确保测试的一致性和可重复性。

回归测试

在软件修改或更新后，重新运行之前的测试用例，确保新的更改没有引入新的问题。

模块化测试

将软件系统划分为多个模块，分别进行测试，有助于定位和解决问题，提高测试质量。

探索性测试

测试人员根据对软件的理解，尝试不同的测试场景和用例，以发现潜在的问题。



软件测试中的常见问题及解决方案

测试用例设计不足

应确保测试用例全面覆盖软件功能，包括正常和异常情况，以提高测试的完整性和有效性。

测试数据管理困难

通过建立有效的测试数据管理机制，如使用测试数据管理工具，可以方便地管理和维护测试数据。

缺陷跟踪和修复问题

建立完善的缺陷跟踪和修复流程，确保每个缺陷都能被及时记录和修复，避免遗漏。

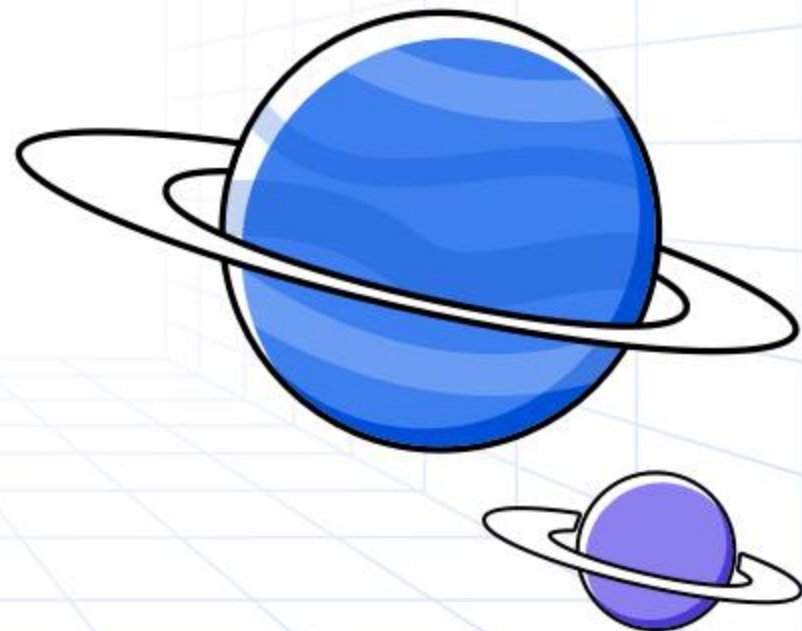
测试与开发的协作问题

加强测试人员与开发人员之间的沟通和协作，确保问题能够及时发现和解决。



08

性能测试优化策略 分析



性能测试指标体系建立

响应时间

衡量系统处理用户请求的速度，包括服务器响应时间和网络传输时间。

并发用户数

衡量系统同时处理多个用户请求的能力，反映系统的并发性能。

吞吐量

衡量系统在单位时间内能够处理的请求数量，反映系统的处理能力。

资源利用率

衡量系统资源（如CPU、内存、磁盘等）的使用情况，反映系统的资源瓶颈。



性能测试场景设计与执行

基准测试

在特定环境下，对系统进行性能测试，
获取系统性能指标。

负载测试

通过模拟实际用户负载，测试系统在
负载情况下的性能指标。

压力测试

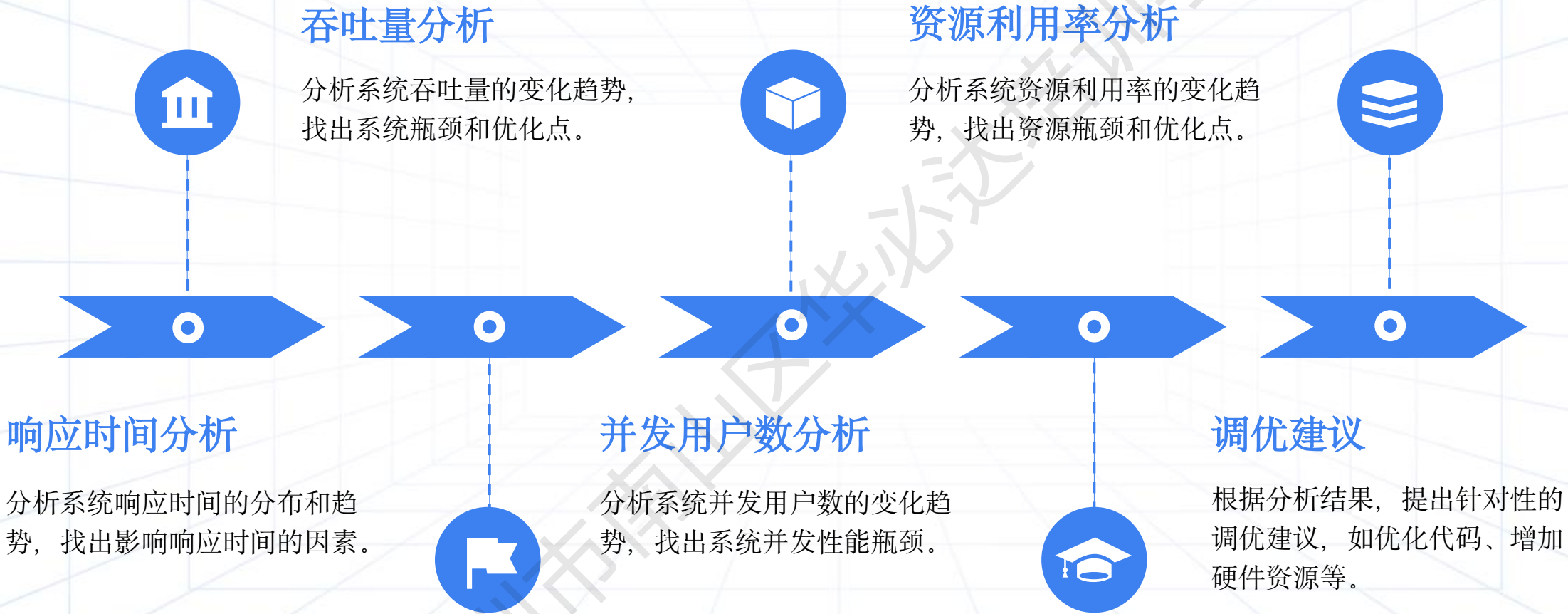
通过逐步增加负载，测试系统在极端
情况下的性能指标和稳定性。

稳定性测试

长时间运行系统，测试系统的稳定性
和可靠性。



性能测试结果分析及调优建议



性能测试工具使用技巧分享

测试工具选择

根据测试需求和测试环境，选择合适的性能测试工具。

测试数据准备

准备符合实际业务场景的测试数据，确保测试结果的准确性和可靠性。

测试脚本编写

编写高效、可复用的测试脚本，提高测试效率和准确性。

测试结果分析

利用测试工具提供的数据分析功能，对测试结果进行深入分析和解读。

总结与展望：未来软件测试发展趋势

01

智能化测试

随着AI技术的发展，未来软件测试将更加智能化，能够自动识别和预测潜在的问题。

02

云测试平台

云测试平台将提供更加灵活和高效的测试环境，支持多种设备和浏览器的测试。

03

持续集成与持续测试

通过持续集成和持续测试，可以更早期地发现和解决问题，提高软件的质量和交付速度。

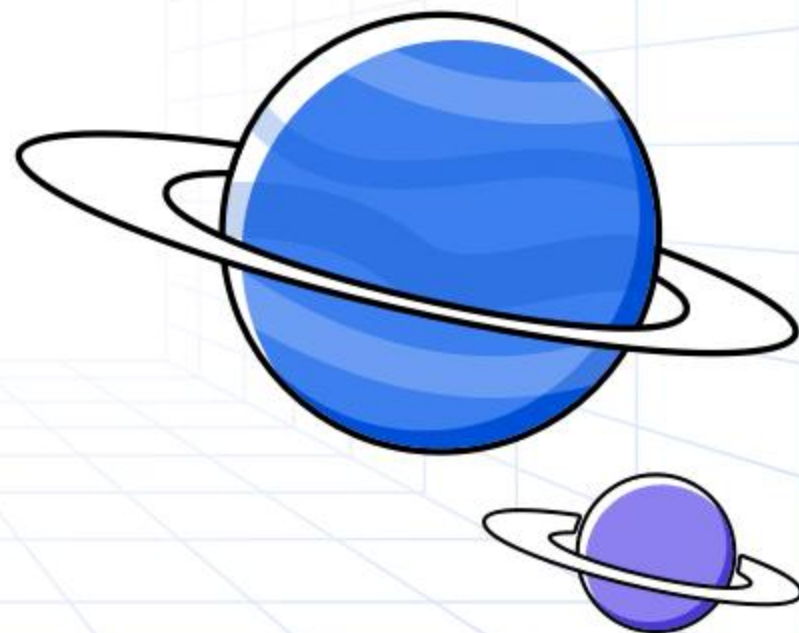
04

测试数据的安全与隐私保护

随着数据安全和隐私保护意识的提高，未来软件测试将更加注重测试数据的安全性和隐私保护。

09

培训与指导



培训



培训体系



培训方法



数据标注员质量提升计划



质检员质量提升计划

指导

01. 实践教学体系构建原则
02. 数据采集流程管理知识
03. 数据处理规范管理知识
04. 数据处理流程管理知识
05. 数据处理工具和设备使用管理知识
06. 数据标注团队的管理

培训与指导考题示例：

1. 质量提升计划周期为技巧提升的质检员需要（ ）质检经验。

- A. 1周以上
- B. 2周以上
- C. 1个月以上
- D. 2个月以上

答案： D

2. 数据标注团队中的职称等级不包括（ ）。

- A. 素材收集员
- B. 高级专家
- C. 教授
- D. 讲师



◆ THANKS ◆
感谢您的观看

